

Hafta 3 — Razor Syntax, View Component'ler, Layout Yapısı ve Kullanımı

Ders: Web Programlama · **Hafta:** 3 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 2 (MVC katmanları, action, `ActionResult`, routing) **Bu dosya sınav kapsamındadır.** Daha derin konular `H03-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Bir `.cshtml` dosyasının **nerede ve ne zaman** çalıştığını **açıklayabileceksiniz**; tarayıcıya ne gittiğini kanıtlayabileceksiniz.
2. `@` ifadelerini, `@{ }` bloklarını, `@if` ve `@foreach` yapılarını **yazabileceksiniz**.
3. `@model` ile **güçlü tipli (strongly typed) görünüm** tanımlayıp controller'dan veri **taşıyabileceksiniz**.
4. Razor'ın **otomatik HTML kodlaması** yaptığını ve `@Html.Raw` 'ın bu korumayı neden kaldırdığını **gösterebileceksiniz**.
5. **Layout** kurup `@RenderBody()` 'nin ne yaptığını **açıklayabileceksiniz**.
6. `_ViewStart.cshtml` ve `_ViewImports.cshtml` dosyalarının görevlerini **ayırta edebileceksiniz**.
7. Bir görünüm bulunamadığında çatının **hangi yolları hangi sırayla** aradığını **sayabileceksiniz**.
8. **Partial View** ile **View Component** arasındaki farkı **açıklayabilecek** ve verilen bir ihtiyaç için doğru olanı **seçebileceksiniz**.

Haftanın ilkesi — Tek doğruluk kaynağı

Aynı arayüz parçası iki yerde duruyorsa, er ya da geç ikisi **farklılaşır**. Üçüncü kopyayı eklediğinizde fark edilmesi aylar alır. Bu haftanın bütün araçları — Layout, Partial View, View Component — tek bir soruya cevap verir: *"Bu parçanın tek bir sahibi kim?"*

1. Razor ve `.cshtml`

Hafta 2'de action'larımız `Content()` ile **düz metin** döndürdü. Bu bir öğretim kolaylığıydı: routing'e odaklanabilmek için HTML'i devre dışı bıraktık. Bu hafta `return View()` diyoruz ve sunucunun HTML'i nasıl **ürettiğine** bakıyoruz.

1.1. Razor nerede çalışır?

Razor bir şablon motorudur. `.cshtml` dosyası HTML ile C#'ın aynı dosyada yazılmasına izin verir. Ama çalıştığı yer kritiktir:

.cshtml sunucuda çalışır. Tarayıcı @ karakterini hiç görmez.

Akış şudur:

#	Nerede	Ne olur
1	Tarayıcı	GET /Kitap/Detay/42 isteği gider
2	Sunucu	Routing → <code>KitapController.Detay(42)</code> çalışır
3	Sunucu	<code>return View(kitap)</code> → <code>Views/Kitap/Detay.cshtml</code> bulunur
4	Sunucu	Razor dosyası önceden derlenmiş bir C# sınıfıdır ; çalıştırılır
5	Sunucu	C# ifadeleri değerlendirilir, geriye düz HTML metni kalır
6	Ağ	Yanıt gövdesinde yalnızca o HTML gider
7	Tarayıcı	HTML'i çizer — <code>@foreach</code> diye bir şey görmemiştir

Bunu kendiniz kanıtlayın: sayfayı açın, **Ctrl+U** (Kaynağı görüntüle) deyin. `@model`, `@foreach`, `@if` göremezsiniz. Yalnızca sonuç HTML vardır.

⚠ Her yıl sorulan yanlış: "Razor tarayıcıda çalışır." Hayır. JavaScript tarayıcıda çalışır; Razor sunucuda. İkisi aynı dosyada yan yana yazılabilir ama **farklı makinelerde, farklı zamanlarda** çalışırlar.

Bunun pratik üç sonucu vardır:

- Razor içinde yazdığınız C# kodu **sunucunun kaynaklarını** kullanır — veritabanına erişebilir (ama erişmemelisiniz, §3.5).
- Razor içindeki bir şeyi tarayıcının geliştirici konsolunda **hata ayıklayamazsınız**; orada yoktur.
- Kullanıcı tıkladığında Razor **yeniden çalışmaz**; yeni bir istek gerekir. Sayfanın bir parçasını tıklamayla değiştirmek JavaScript'in işidir.

1.2. @ — tek kural

Razor'da ezberlenecek tek kural şudur:

@ = "buradan sonrası C#".

```
<h1>@kitap.Baslik</h1>
<p>Fiyat: @kitap.Fiyat TL</p>
<p>Bugün: @DateTime.Now.ToShortDateString()</p>
```

Razor ifadenin nerede bittiğini **kendi anlar**: `@kitap.Baslik` yazdığınızda sonraki `<` karakterinde C# biter, HTML başlar.

Karışık bir durumda parantez kullanılır — buna **açık ifade (explicit expression)** denir:

```
<p>@(kitap.Fiyat * 0.82m) TL (KDV hariç)</p>
<p>@(kitap.Baslik)-indirimli.pdf</p>    @* parantez olmasaydı "-indirimli.pdf" C# sanılırdı *@
```

Gerçek bir @ yazmak için iki kez yazılır:

```
<p>İletişim: baran.kaynak@@ornek.edu.tr</p>
```

Razor yorumu @* ... *@ ile yazılır ve **tarayıcıya hiç gitmez**:

```
@* Bu satır HTML çıktısında YOKTUR *@
<!-- Bu satır HTML çıktısında VARDIR; kullanıcı Ctrl+U ile görür -->
```

💡 Parolayı, iç notu, "TODO"yu HTML yorumuna yazmayın — kullanıcı görür. Razor yorumuna yazın.

1.3. Kod blokları ve denetim yapıları

Birden çok C# satırı gerekiyorsa @{ ... } kullanılır. Bu blok **hiçbir şey yazdırmaz**; yalnızca çalışır:

```
@{
    var indirimli = kitap.Fiyat * 0.9m;
    var etiket = indirimli < 50 ? "ucuz" : "normal";
}
<p class="@etiket">@indirimli TL</p>
```

Denetim yapıları @ ile başlar, gövdeleri **HTML yazabilir**:

```
@if (Model.Count == 0)
{
    <p class="bos">Hiç kitap yok.</p>
}
else
{
    <ul>
        @foreach (var k in Model)
        {
            <li>@k.Baslik – @k.Fiyat TL</li>
        }
    </ul>
}
```

Dikkat edilecek üç nokta:

Kural	Örnek
@if 'ten sonra gelen else / else if @ almaz	@if (...) { } else { }

Blok içinde HTML etiketi açıp kapatmak serbesttir	<code>@k.Baslik</code>
Etiketsiz düz metin yazacaksanız <code>@:</code> ya da <code><text></code> gerekir	<code>@: Sonuç yok</code>

```
@foreach (var k in Model)
{
    @:- @k.Baslik          @* etiketsiz satır *@
    <text>@k.Fiyat TL</text> @* çok satırlı etiketsiz blok *@
}
```

1.4. @model — güçlü tipli görünüm

Controller veriyi `View(...)` ile taşır; görünüm onu `Model` adıyla alır:

```
// Controllers/KitapController.cs
public IActionResult Detay(int? id)
{
    var kitap = _depo.Bul(id);
    if (kitap is null) return NotFound();
    return View(kitap);          // ← model nesnesi
}
```

```
@* Views/Kitap/Detay.cshtml *@
@model KampusKitap.Models.Kitap

<h1>@Model.Baslik</h1>
<p>@Model.Yazar – @Model.Fiyat TL</p>
```

Yazım	Anlamı
<code>@model</code> (küçük m)	Yönerge. Bu görünümün model tipini bildirir. Dosyada bir kez , en üstte.
<code>@Model</code> (büyük M)	Nesne. Controller'ın gönderdiği gerçek veri.

Güçlü tipli olmanın karşılığı derleme zamanı güvenlidir. `@Model.Baslik` yazarsanız sayfa açılmadan hata alırsınız; `ViewBag.Baslik` yazsaydınız sessizce `null` gelirdi.

Liste taşımak için tip liste olur:

```
@model List<KampusKitap.Models.Kitap>
<p>@Model.Count kitap bulundu.</p>
```

⚠ `@model` yönergesinde **tam ad** (namespace ile) gerekir — ama `_ViewImports.cshtml` içinde `@using KampusKitap.Models` varsa kısa yazabilirsiniz (§2.4).

1.5. Otomatik HTML kodlaması — ve @Html.Raw

Razor, @ifade ile yazdığı **her değeri otomatik olarak HTML-kodlar**. Bu bir kolaylık değil, **güvenlik önlemidir**.

Bir kullanıcının kitap başlığına şunu yazdığını düşünün:

```
<script>alert('ele geçirildi')</script>
```

```
<h1>@Model.Baslik</h1>
```

Tarayıcıya giden HTML:

```
<h1>&lt;script&gt;alert('ele geçirildi')&lt;/script&gt;</h1>
```

Ekranda **metin olarak** görünür. Script **çalışmaz**. Razor < karakterini < yaptı.

Şimdi korumayı kaldıralım:

```
<h1>@Html.Raw(Model.Baslik)</h1>
```

Tarayıcıya giden HTML:

```
<h1><script>alert('ele geçirildi')</script></h1>
```

Script çalışır. Bu saldırıya **XSS (Cross-Site Scripting — siteler arası betik çalıştırma)** denir. Saldırgan sizin sayfanızda kendi kodunu çalıştırır: oturum çerezini okuyabilir, form gönderebilir, sayfayı değiştirebilir.

 **Kural:** @Html.Raw yazarken "bu HTML'i ben ürettim, kullanıcıdan gelmedi" demiş olursunuz. Kullanıcıdan gelen hiçbir veride kullanılmaz.

Yazım	Çıktı	Ne zaman
@Model.Baslik	kodlanmış — güvenli	Varsayılan. Her zaman.
@Html.Raw(x)	ham HTML	Yalnızca kaynağına güvendiğiniz HTML'de

1.6. @Html yardımcıları ve Tag Helper köprüsü

@Html.Raw gördüğünüz @Html nesnesinin bir üyesidir. @Html ASP.NET'in eski sürümlerinden gelir ve HTML üretmek için metotlar sunar:

```
@Html.TextBoxFor(m => m.Baslik)
@Html.ActionLink("Detay", "Detay", new { id = 42 })
```

Bu yazım **çalışır** ama okunması zordur: HTML'e benzemez. ASP.NET Core bunun yerine **Tag Helper**'ları getirdi — aynı işi HTML etiketi gibi görünen bir yazımla yapar:

```
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="42">Detay</a>
```



Hafta 5 tamamen Tag Helper'lara ayrılmıştır. Bu hafta yalnızca `@Html` 'in ne olduğunu ve `@Html.Raw` 'ı biliyoruz; form üretimini Hafta 4–5'te öğreneceğiz.

2. Layout ve Paylaşılan Yapı

2.1. Problem: tekrar eden arayüz

Üç sayfanız olsun: kitap listesi, kitap detayı, kategori listesi. Üçünde de aynı şeyler var:

- `<!DOCTYPE html>`, `<head>`, CSS bağlantıları
- Üst menü
- Alt bilgi (footer)
- Script etiketleri

Bunu üç dosyaya kopyalarsanız: menüye bir bağlantı eklemek **üç dosya** düzenlemek demektir. Dördüncü sayfa eklendiğinde dördü. Birini unuttuğunuzda **sessizce tutarsız** bir site olur.

✖ Haftanın ilkesi tam burada: **tek doğruluk kaynağı**.

2.2. Layout ve `@RenderBody()`

Layout, sayfanın **değişmeyen** kısmını tutan bir `.cshtml` dosyasıdır. Değişen kısmın geleceği yeri `@RenderBody()` ile işaretler:

```
@* Views/Shared/_Layout.cshtml *@
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="utf-8" />
  <title>@ViewData["Title"] - KampusKitap</title>
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>
    <nav>
      <a href="/">Ana sayfa</a>
      <a href="/Kitap">Kitaplar</a>
      <a href="/Kategori">Kategoriler</a>
    </nav>
  </header>

  <main>
    @RenderBody()          @* ← sayfaya özel içerik TAM BURAYA gelir *@
  </main>

  <footer>© 2026 KampusKitap</footer>

  <script src="~/js/site.js"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

@RenderBody() nedir? Çalıştırılan görünümün (örn. `Views/Kitap/Index.cshtml`) ürettiği HTML'in layout'un içine **yerleştirildiği noktadır**. Layout'ta **tam bir kez** bulunmak zorundadır.

Sıralama şöyledir — ve çoğu kişinin beklediğinin **tersidir**:

#	Çalışan	Not
1	Görünüm (<code>Index.cshtml</code>)	Önce içerik üretilir
2	Layout (<code>_Layout.cshtml</code>)	Sonra çerçeve çalışır
3	<code>@RenderBody()</code>	1. adımın çıktısı buraya yapıştırılır

Bu sıra, `@section` 'ların neden çalıştığını açıklar (§2.6): görünüm, layout'tan önce çalıştığı için kendi parçalarını layout'a **önceden** bırakabilir.

Adlandırma sözleşmesi: layout ve partial dosyaları **alt çizgi ile** başlar (`_Layout.cshtml` , `_ViewStart.cshtml`). Alt çizgi "bu dosya tek başına bir sayfa değildir" demektir. Çatıyı bağlamaz; **okuyan insanı** bağlar.

2.3. `_ViewStart.cshtml` — layout'un otomatik atanması

Her görünümün başına `Layout = "_Layout";` yazmak yine tekrar olurdu. `_ViewStart.cshtml` bunu çözer: **her görünümünden önce çalışır**.

```
@* Views/_ViewStart.cshtml *@
@{
    Layout = "_Layout";
}
```

- `Views/` köündeki `_ViewStart.cshtml` **tüm** görünüm için çalışır.
- `Views/Kitap/_ViewStart.cshtml` eklerseniz, **yalnızca** o klasör için ek olarak çalışır (kökteki de çalışmaya devam eder, önce o çalışır).
- Tek bir görünümde layout istemiyorsanız o görünümde `Layout = null;` yazarsınız (§2.7).

2.4. `_ViewImports.cshtml` — ortak yönergeler

Her görünüme `@using KampusKitap.Models` yazmak da tekrardır. `_ViewImports.cshtml` **yönergeleri** paylaşır:

```
@* Views/_ViewImports.cshtml *@
@using KampusKitap
@using KampusKitap.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Yönerge	İşi
<code>@using</code>	Namespace'i tüm görünümlere açar → <code>@model Kitap</code> yeterli olur
<code>@addTagHelper</code>	Tag Helper'ları etkinleştirir (Hafta 5)
<code>@inject</code>	Bir servisi tüm görünümlere enjekte eder (§3.5 uyarısıyla)

⚠ **İkisini karıştırmayın.** `_ViewStart` **kod** çalıştırır (`Layout = ...`). `_ViewImports` **yönerge** paylaşır (`@using` , `@addTagHelper`). Birinin işini diğerine yazarsanız çalışmaz.

2.5. Görünüm arama sırası

`return View()` dediğinizde çatı dosyayı **sırayla** arar. Bu sırayı bilmek, bu haftanın en pratik kazanımıdır — çünkü hata mesajını okumanızı sağlar.

`KitapController.Index()` içinde `return View();` için arama sırası:

#	Aranan yol
1	<code>/Views/Kitap/Index.cshtml</code>
2	<code>/Views/Shared/Index.cshtml</code>

`return View("Liste")` dersiniz `Index` yerine `Liste` aranır. `return View("~/Views/Baska/X.cshtml")` gibi **tam yol** vererseniz arama hiç yapılmaz.

İkinci satır (`Shared`) önemlidir: **bütün controller'ların ortak kullandığı görünüm** oraya konur.

`Error.cshtml` tam olarak böyle çalışır.

Bulunamazsa çatı susmaz: aradığı **bütün yolları listeleyen** bir istisna fırlatır. O listeyi okumak çözümün kendisidir (🐛 Hata Köşesi).

2.6. @section ve @RenderSection

Bazen sayfaya özel bir şeyin layout'un **başka bir yerine** girmesi gerekir — tipik örnek, yalnızca bir sayfada gereken script'in `</body>` 'den hemen önce yer alması.

Layout'ta yer ayrılır:

```
@await RenderSectionAsync("Scripts", required: false)
```

Görünümde doldurulur:

```
@* Views/Kitap/Detay.cshtml *@
@section Scripts {
    <script src="~/js/kitap-detay.js"></script>
}
```

Yazım	Anlamı
<code>required: false</code>	Bölüm doldurulmazsa sorun yok , boş geçilir
<code>required: true</code>	Bölümü doldurmayan her görünüm hata verir
<code>@RenderSection("X", false)</code>	Eş anlamlı eski (senkron) yazım

🐛 `@section Scripts` yazdınız ama layout'ta `RenderSection("Scripts", ...)` yoksa: *"The following sections have been defined but have not been rendered"* hatası alırsınız. Bölüm adları **büyük/küçük harfe duyarlıdır**.

2.7. Birden fazla layout — ve hiç layout olmaması

Bir projede birden çok layout olabilir: `_Layout.cshtml` (site), `_AdminLayout.cshtml` (yönetim). Seçim görünümün başında yapılır:

```
@{
    Layout = "_AdminLayout";
}
```

Hiç çerçeve istemiyorsanız (örn. yazdırma sayfası, e-posta şablonu):

```
@{
    Layout = null;
}
```

3. Partial View ve View Component

Layout **sayfanın çerçevesini** paylaşır. Peki sayfanın **ortasındaki** tekrar eden parça? Kitap kartı listede, arama sonucunda ve ana sayfada aynı görünecekse?

3.1. Partial View

Partial View, **yeniden kullanılabilir bir görünüm parçasıdır**. Kendi layout'u yoktur, tek başına bir sayfa değildir; adı alt çizgiyle başlar.

```
@* Views/Shared/_KitapKart.cshtml *@
@model KampusKitap.Models.Kitap

<article class="kart">
    <h3>@Model.Baslik</h3>
    <p>@Model.Yazar</p>
    <p class="fiyat">@Model.Fiyat TL</p>
</article>
```

Kullanımı:

```
@foreach (var k in Model)
{
    <partial name="_KitapKart" model="k" />
}
```

`<partial>` bir **Tag Helper**'dir (Hafta 5). Eski yazımı `@await Html.PartialAsync("_KitapKart", k)` şeklindedir; aynı işi yapar.

Kritik nokta: partial **veriyi kendisi getirmez**. Üst görünüm ne verirse onu basar. `model="k"` yazmazsanız partial üst görünümün modelini alır — ve tip uyuşmazsa hata verir.

3.2. View Component

Bazı parçaların **kendi verisi** vardır ve bu veri sayfanın modeliyle ilgili değildir:

- Üst menüdeki sepet sayısı
- Kenar çubuğundaki "en çok okunan 5 kitap"
- Sayfa altındaki ziyaret sayacı

Bunu partial ile yapmaya kalkarsanız, veriyi **o parçayı kullanan her controller'ın** hazırlaması gerekir. Beş sayfada kullanıyorsanız beş controller aynı kodu yazar — yine tekrar, yine tek doğruluk kaynağı ihlali.

View Component bu sorunu çözer: hem mantığı hem görünümü kendisi taşır.

İki parçası vardır.

1 — Sınıf (`ViewComponents/ZiyaretRozetiViewComponent.cs`):

```
using Microsoft.AspNetCore.Mvc;
using KampusKitap.Services;

namespace KampusKitap.ViewComponents;

public class ZiyaretRozetiViewComponent : ViewComponent
{
    private readonly IZiyaretSayaci _sayac;

    // Hafta 2'de kurduğumuz DI aynen çalışıyor.
    public ZiyaretRozetiViewComponent(IZiyaretSayaci sayac)
    {
        _sayac = sayac;
    }

    public IViewComponentResult Invoke()
    {
        var deger = _sayac.Arttir(); // ← VERİYİ KENDİ GETİRİYOR
        return View(deger);         // ← Default.cshtml
    }
}
```

2 — Görünüm (Views/Shared/Components/ZiyaretRozeti/Default.cshtml):

```
@model int
<span class="rozet">Bu sayfa @Model kez gösterildi</span>
```

Kullanımı — **herhangi bir görünümünden, hatta layout'tan:**

```
<vc:ziyaret-rozeti />
```

Eski yazım: `@await Component.InvokeAsync("ZiyaretRozeti")`.

⚠️ Klasör yolu sözleşmedir, isteğe bağlı değil:

Views/Shared/Components/{BileşenAdı}/Default.cshtml Sınıf adındaki `ViewComponent` soneki **atılır**:
`ZiyaretRozetiViewComponent` → `ZiyaretRozeti`. `<vc:...>` etiketinde ad **kebab-case'e** çevrilir:
`ZiyaretRozeti` → `ziyaret-rozeti`.

Veri alan ve asenkron çalışan sürüm `InvokeAsync` ile yazılır:

```
public async Task<IViewComponentResult> InvokeAsync(int adet)
{
    var kitaplar = await _depo.EnCokOkunanlarAsync(adet);
    return View(kitaplar);
}
```

```
<vc:populer-kitaplar adet="5" />
```

3.3. Partial ↔ View Component — tek cümlelik fark

Partial veriyi alır. View Component veriyi kendi getirir.

	Partial View	View Component
Veriyi nereden alır	Üst görünümünden (<code>model="..."</code>)	Kendi <code>Invoke</code> / <code>InvokeAsync</code> içinde
Kendi C# sınıfı var mı	Yok	Var (<code>: ViewComponent</code>)
DI ile servis alabilir mi	Doğrudan hayır	Evet (kurucu enjeksiyonu)
Controller'a bağımlı mı	Evet — veriyi controller hazırlar	Hayır — bağımsız çalışır
HTTP isteği alır mı	Hayır	Hayır (controller değildir, route'lanmaz)
Tipik kullanım	Kitap kartı, form alanı, tablo satırı	Sepet sayısı, menü, "popüler" listesi
Çağırma	<code><partial name="_X" model="y" /></code>	<code><vc:x /></code>

 **Sınavda ayırt etme sorusu gelir.** Kendinize şunu sorun: "Bu parçayı yeni bir sayfaya koyduğumda, o sayfanın controller'ında bir şey yazmam gerekiyor mu?" **Gerekliyse** partial'dır (ya da View Component olmalıdır). **Gerekmiyorsa** View Component'tir.

3.4. Karar tablosu

Durum	Doğru araç
Her sayfada aynı çerçeve (menü, footer)	Layout
Sayfanın modelinden beslenen tekrar eden parça	Partial View
Kendi verisini getiren, sayfadan bağımsız parça	View Component
Kendi adresi (URL) olması gereken içerik	Ayrı action + görünüm
Tıklamayla sunucuya gidip sayfanın bir parçasını yenileme	Ayrı action + partial döndürme (Hafta 12 ile birlikte)

3.5. Görünüm içinde iş mantığı yazmanın bedeli

Razor sunucuda çalıştığı için görünüm içinden veritabanına **erişebilirsiniz**:

```
@inject KampusKitap.Data.KitapDeposu Depo
@{
    var kitaplar = Depo.Hepsi();    @* ÇALIŞIR — ama yapmayın *@
}
```

Çalışır. Bedeli:

1. **Test edilemez.** Görünüm bir HTML şablonudur; birim testi yazılmaz.

2. **Hata yönetimi yoktur.** Sorgu patlarsa kullanıcı yarı çizilmiş bir sayfa görür — HTTP başlıkları çoktan gönderilmiştir.
3. **Gizli N+1.** Döngü içinde sorgu yazmak kolaydır; 50 satırlık liste 51 sorgu yapar ve kimse fark etmez.
4. **Sorumluluk dağılır.** "Bu veri nereden geliyor?" sorusunun cevabı artık tek yerde değildir.

`@inject` yasak değildir — yerelleştirme servisi, yapılandırma okuma gibi **görüntülemeye ait** şeyler için meşrudur. Yasak olan **veri getirmektir**.

Hata Köşesi — Hafta 3

Belirti	Gerçek sebep	Çözüm
<code>InvalidOperationException: The view 'Index' was not found</code> + yol listesi	Dosya adı/klasörü sözleşmeye uymuyor	Listedeki yolları okuyun ; dosyanız tam olarak o yollardan birinde mi?
Sayfa açılıyor ama menü/ çerçeve yok	<code>Layout = null</code> ya da <code>_ViewStart.cshtml</code> yanlış klasörde	<code>_ViewStart.cshtml</code> <code>Views/</code> kökünde mi?
<code>The following sections have been defined but have not been rendered: 'Scripts'</code>	Görünümde <code>@section Scripts</code> var, layout'ta <code>RenderSection</code> yok	Layout'a <code>@await RenderSectionAsync("Scripts", required: false)</code> ekleyin
<code>@Model.Baslik</code> → <code>NullReferenceException</code>	Controller <code>View()</code> dedi, <code>View(model)</code> demedi	Modeli gönderin ya da görünümde <code>@model</code> kaldırın
Ekranda <code>&lt;b&gt;</code> gibi metin çıkıyor	Veri zaten HTML kodlanmış, bir kez daha kodlanıyor	Veriyi kaynağında ham tutun; kodlamayı Razor yapsın
<code><vc:ziyaret-rozeti /></code> olduğu gibi ekranda yazıyor	<code>_ViewImports.cshtml</code> 'de <code>@addTagHelper *, KampusKitap</code> yok	Projenin kendi assembly'sini Tag Helper olarak ekleyin
View Component "görünüm bulunamadı" diyor	Klasör yolu yanlış	<code>Views/Shared/Components/{Ad}/Default.cshtml</code> — <code>Ad</code> 'da <code>ViewComponent</code> soneki yok
<code>.cshtml</code> değişti, sayfa eski hâlini gösteriyor	Tarayıcı ön belleği ya da uygulama yeniden başlamadı	Ctrl+F5; gerekirse <code>dotnet run</code> 'ı yeniden başlatın

Klasik Tuzak — Görünümün içine veri sorgusu yazmak

Kod çalışır. Test geçmez; çünkü test yoktur.

```
@* Views/Kitap/Index.cshtml – "sadece bu sefer" *@
@inject KampusKitap.Data.KitapDeposu Depo

@foreach (var k in Depo.Hepsi())
{
    <article>
        <h3>@k.Baslik</h3>
        @* ve döngünün içinde ikinci bir sorgu... *@
        <p>@Depo.KategoriAdi(k.KategoriId)</p>
    </article>
}
```

Neden tuzak: hiçbir aşamada hata vermez. Sayfa açılır, liste gelir, her şey doğru görünür. 50 kitaplı listede **51 sorgu** çalışır. Geliştirme makinesinde fark edilmez; 5.000 kayıta sayfa 8 saniyede açılır ve sebebi aramak günler alır.

Doğrusu: veri controller'da (ya da servis katmanında) **bir kez** alınır, görünümüne hazır olarak verilir. Görünüm yalnızca **gösterir**.

```
public IActionResult Index()
{
    var liste = _depo.HepsiKategoriyleBirlikte(); // tek sorgu
    return View(liste);
}
```

Bu tuzağın adı **N+1 sorgu problemidir** ve Hafta 8'de EF Core'da aynıyla karşılaşacağız. Orada çözümü **Include** olacak; burada çözümü **sorguyu görünümünden çıkarmak**.

Terimler Sözlüğü

Terim	Türkçe karşılığı (ilk geçiş)	Ne demek
Razor	—	HTML + C# şablon motoru; .cshtml dosyalarını işler
View	Görünüm	Kullanıcının göreceği HTML'i üreten .cshtml dosyası
Layout	Yerleşim	Sayfanın değişmeyen çerçevesi
@RenderBody()	—	Görünüm çıktısının layout içine yerleştiği nokta
Section	Bölüm	Görünümün layout'ta ayrılmış bir yere bıraktığı parça
Partial View	Kısmi görünüm	Veriyi dışarıdan alan, yeniden kullanılabilir görünüm parçası
View Component	Görünüm bileşeni	Kendi verisini getiren, kendi sınıfı olan görünüm parçası
Strongly typed view	Güçlü tipli görünüm	@model ile tipi bildirilmiş görünüm
HTML encoding	HTML kodlaması	< → &lt; dönüşümü; XSS'in temel savunması

XSS	Siteler arası betik çalıştırma	Saldırganın kodunun sizin sayfanızda çalışması
N+1 problemi	—	Bir liste sorgusu + her satır için bir sorgu daha

✓ Kontrol Soruları

A. Razor temelleri

1. Bir `.cshtml` dosyası nerede çalışır? Tarayıcıya giden yanıtta `@foreach` neden yoktur?
2. `@model` ile `@Model` arasındaki fark nedir?
3. Aşağıdaki satır ekranda ne gösterir? `<p>baran@ornek.edu.tr</p>`
4. `@* ... *@` ile `<!-- ... -->` arasındaki fark nedir? Hangisine iç not yazılır?
5. `@(kitap.Fiyat * 2)` yazımındaki parantez neden gerekli olabilir?

B. Kodlama ve güvenlik

6. `@Model.Aciklama` ile `@Html.Raw(Model.Aciklama)` arasındaki fark nedir?
7. Bir kullanıcı yorumu alanına `<img src=x onerror=alert(1)>` yazdı. İki yazımda da ne olur?
8. `@Html.Raw` hangi durumda meşrudur?

C. Layout

9. `@RenderBody()` ne yapar? Bir layout'ta kaç kez bulunabilir?
10. Görünüm mü önce çalışır, layout mu? Bu sıra neden önemlidir?
11. `_ViewStart.cshtml` ile `_ViewImports.cshtml` arasındaki fark nedir? Hangisine `@using` yazılır?
12. `@section Scripts { ... }` yazdınız ama layout'ta karşılığı yok. Ne olur?
13. Tek bir sayfada çerçeve istemiyorsanız ne yazarsınız?

D. Görünüm arama

14. `KategoriController.Liste()` içinde `return View();` yazdınız. Çatı hangi iki yolu bu sırayla arar?
15. `return View("Detay")` dersiniz arama nasıl değişir?
16. Bir görünümü **tüm** controller'ların kullanmasını istiyorsanız nereye koyarsınız?

E. Partial ve View Component

17. "Partial veriyi ..., View Component veriyi ..." — cümleyi tamamlayın.
18. Ana sayfada, kitap listesinde ve kategori sayfasında görünen bir **sepet sayısı** rozeti yazacaksınız. Hangisini seçersiniz, neden?
19. Bir kitap kartını üç farklı listede göstereceksiniz; veri her üçünde de sayfanın modelinde var. Hangisi?
20. `PopulerKitaplarViewComponent` sınıfının görünümü hangi tam yolda olmalıdır?
21. View Component'e route ile doğrudan erişilebilir mi? Neden?
22. Görünüm içine veritabanı sorgusu yazmak neden hata vermediği hâlde yanlıştır? İki sebep yazın.

Kaynakça

Her kaynağın yanında **bu ders notunun hangi bölümünü** dayandırdığı yazılıdır. Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür. **Örnekte** `Startup.cs` **görüyorsanız yanlış sürümdesiniz**.

Zorunlu okuma

Kaynak	Dayandırdığı bölüm
Razor syntax reference for ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/razor?view=aspnetcore-10.0	§1.2, §1.3 — <code>@</code> , <code>@{ }</code> , <code>@@</code> , <code>@:</code> , <code><text></code> , Razor yorumu
Views in ASP.NET Core MVC — https://learn.microsoft.com/aspnet/core/mvc/views/overview?view=aspnetcore-10.0	§1.1, §1.4, §2.5 — görünümün rolü, <code>@model</code> , arama sırası
Layout in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/layout?view=aspnetcore-10.0	§2.2, §2.3, §2.4, §2.6 — <code>@RenderBody</code> , <code>_ViewStart</code> , <code>_ViewImports</code> , section
Partial views in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/partial?view=aspnetcore-10.0	§3.1 — partial, <code><partial></code> Tag Helper, <code>PartialAsync</code>
View components in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/view-components?view=aspnetcore-10.0	§3.2, §3.3 — <code>ViewComponent</code> , <code>InvokeAsync</code> , klasör sözleşmesi, <code><vc:></code>

Güvenlik (§1.5)

Kaynak	Dayandırdığı bölüm
Prevent Cross-Site Scripting (XSS) in ASP.NET Core — https://learn.microsoft.com/aspnet/core/security/cross-site-scripting?view=aspnetcore-10.0	§1.5 — otomatik kodlama, <code>Html.Raw</code> riski
OWASP — Cross Site Scripting Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html	§1.5 — saldırının neden ciddi olduğu, bağlam bazlı kodlama

Köprüler (sonraki haftalara)

Kaynak	Dayandırdığı bölüm
Tag Helpers in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro?view=aspnetcore-10.0	§1.6, §3.1 — <code>@Html</code> → Tag Helper geçişi (Hafta 5)
Razor file compilation in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/view-compilation?view=aspnetcore-10.0	§1.1 — <code>.cshtml</code> 'in derlenmesi (ayrıntısı <code>H03-detay.md</code>)

Daha Fazlası

`H03-detay.md` dosyasında (sınav kapsamı dışı):

- Razor dosyası hangi C# sınıfına dönüşür — üretilen kodun iskeleti
 - Çalışma zamanı derleme (runtime compilation) ve ne zaman açılır
 - `IViewLocationExpander` ile arama sırasını değiştirmek
 - `ViewComponent` sonucunun `Content`, `View`, `ViewComponentResult` varyantları
 - Partial ile `RenderPartialAsync` farkı ve neden `PartialAsync` tercih edilir
 - Razor Pages'in `.cshtml` dosyalarıyla ilişkisi
-

Haftaya Hazırlık

Bu hafta ekrana **veri yazdırmayı** öğrendik — tek yön: sunucudan kullanıcıya. **Hafta 4**'te ters yöne geçiyoruz: **kullanıcıdan sunucuya veri almak**.

Formu göster → kullanıcı doldur → gönder → sunucu C# nesnesine çevir → işle.

Düşünerek gelin: Hafta 1'de bir HTTP isteğinin **gövdesi (body)** olduğunu görmüştük. Bir form "Gönder" dendiğinde tarayıcı gövdeye tam olarak ne koyar? Ve sunucu o metni nasıl olup da bir `Kitap` nesnesine çeviriyor?